

# Taller Intensivo: Sistema de Inventario Profesional IUJO

**Framework:** Django 6.0 + Bootstrap 5

**Institución:** IUJO Barquisimeto

**Objetivo:** Crear un sistema CRUD seguro con autenticación y API REST.

---

## 📁📁📁 PARTE 1: Configuración e Instalación (Sesión 1)

### 1.1 Entorno Virtual y Dependencias

Ejecuta estos comandos en tu terminal para aislar el proyecto:

```
# Crear y activar entorno
python -m venv venv
# Windows:
.\venv\Scripts\activate

# Instalación de librerías necesarias
pip install django djangorestframework
```

### 1.2 Estructura del Proyecto

```
django-admin startproject sistema_iujo .
python manage.py startapp inventario
```

En `sistema_iujo/settings.py`, registra las aplicaciones y configura las redirecciones de seguridad (Pág. 9 de la presentación):

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Mis Apps
    'inventario',
    'rest_framework',
]

# Configuración de Redirección (Seguridad)
LOGIN_REDIRECT_URL = 'lista_productos'
```

```
LOGOUT_REDIRECT_URL = 'login'
```

---

## 📁📁 PARTE 2: El Modelo de Datos (MVT - Model)

### 2.1 Definición en `inventario/models.py`

```
from django.db import models

class Categoria(models.Model):
    nombre = models.CharField(max_length=100)
    def __str__(self): return self.nombre

class Producto(models.Model):
    nombre = models.CharField(max_length=200)
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField(default=0)
    categoria = models.ForeignKey(Categoria, on_delete=models.CASCADE)

    def __str__(self): return self.nombre
```

#### Sincronizar base de datos:

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser # Crea tu usuario aquí
```

---

## 📁📁 PARTE 3: Seguridad y Autenticación (Sesión 2)

### 3.1 Formulario de Producto (`inventario/forms.py`)

```
from django import forms
from .models import Producto

class ProductoForm(forms.ModelForm):
    class Meta:
        model = Producto
        fields = ['nombre', 'precio', 'stock', 'categoria']
```

### 3.2 Lógica de Negocio con Protección (`inventario/views.py`)

```
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from .models import Producto
from .forms import ProductoForm
```

```
def lista_productos(request):
    productos = Producto.objects.all()
    return render(request, 'inventario/lista.html', {'productos': productos})

@login_required # Protección nativa (Pág. 9 presentación)
def crear_producto(request):
    if request.method == 'POST':
        form = ProductoForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('lista_productos')
    else:
        form = ProductoForm()
    return render(request, 'inventario/formulario.html', {'form': form})
```

### 3.3 Configuración de URLs (sistema\_iujo/urls.py)

```
from django.contrib import admin
from django.urls import path, include
from inventario import views

urlpatterns = [
    path('admin/', admin.site.urls),
    # Sistema de Autenticación integrado
    path('accounts/', include('django.contrib.auth.urls')),
    # Rutas de la App
    path('', views.lista_productos, name='lista_productos'),
    path('nuevo/', views.crear_producto, name='crear_producto'),
]
```

---

## ▣▣ PARTE 4: La Capa de Presentación (MVT - Template)

### 4.1 Plantilla Base (inventario/templates/inventario/base.html)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="st
    <title>IUJO Inventario</title>
</head>
<body>
    <nav class="navbar navbar-dark bg-primary mb-4">
        <div class="container">
            <a class="navbar-brand" href="/">IUJO Sistema</a>
            <div>
                {% if user.is_authenticated %}
                    <span class="text-white me-2">▣▣ {{ user.username }}</span>
                    <form action="{% url 'logout' %}" method="post" style="display:inline;">
                        {% csrf_token %}
                        <button class="btn btn-sm btn-danger">Salir</button>
                    </form>
                {% else %}
                    <a href="{% url 'login' %}" class="btn btn-sm btn-light">Entrar</a>
```

```

        {% endif %}
    </div>
</div>
</nav>
<div class="container"> {% block content %} {% endblock %} </div>
</body>
</html>

```

## 4.2 Pantalla de Login (inventario/templates/registration/login.html)

```

{% extends 'inventario/base.html' %}
{% block content %}
<div class="col-md-4 mx-auto">
    <div class="card shadow">
        <div class="card-body">
            <h3>Iniciar Sesión</h3>
            <form method="post">
                {% csrf_token %}
                {{ form.as_p }}
                <button type="submit" class="btn btn-primary w-100">Ingresar</button>
            </form>
        </div>
    </div>
</div>
{% endblock %}

```

## 4.3 Lista de Productos (inventario/templates/inventario/lista.html)

```

{% extends 'inventario/base.html' %}
{% block content %}
    <div class="d-flex justify-content-between">
        <h2>Inventario de Productos</h2>
        <a href="{% url 'crear_producto' %}" class="btn btn-success"> + Nuevo </a>
    </div>
    <table class="table mt-3">
        <thead>
            <tr><th>Nombre</th><th>Precio</th><th>Stock</th></tr>
        </thead>
        <tbody>
            {% for p in productos %}
            <tr>
                <td>{{ p.nombre }}</td>
                <td>${{ p.precio }}</td>
                <td><span class="badge {% if p.stock < 5 %}bg-danger{% else %}bg-success{% endif %} {{ p.stock }}</span></td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
{% endblock %}

```

En inventario/serializers.py:

```
from rest_framework import serializers
from .models import Producto

class ProductoSerializer(serializers.ModelSerializer):
    class Meta:
        model = Producto
        fields = '__all__'
```

En sistema\_iujo/urls.py (Añadir al final):

```
from rest_framework import routers
from inventario.serializers import ProductoSerializer
from rest_framework import viewsets

class ProductoViewSet(viewsets.ModelViewSet):
    queryset = Producto.objects.all()
    serializer_class = ProductoSerializer

router = routers.DefaultRouter()
router.register(r'api/productos', ProductoViewSet)

urlpatterns += [path('', include(router.urls))]
```

---

## ☐☐ Resumen de Comandos para Iniciar:

1. `python manage.py runserver`
2. Ir a `http://127.0.0.1:8000/admin` para crear categorías.
3. Ir a `http://127.0.0.1:8000/` para ver el sistema.
4. Probar `http://127.0.0.1:8000/nuevo/` (Pedirá login automáticamente).

**¿Te gustaría que añada el código para la validación de stock negativo en el formulario?**